```
  def director
    get "director"
  end

  def director=(value)
    set "director", value
  end
end
```

A Movie has two methods for each attribute: a reader, such as Movie#title, and a writer, such as Movie#title=. Bob can now load a new movie into the database by firing up the Ruby interactive interpreter and typing the following:

```
movie = Movie.new(1)
movie.title = "Doctor Strangelove"
movie.director = "Stanley Kubrick"
```

This code creates a new record in movies, which has values 1, Doctor Strangelove, and Stanley Kubrick for the columns id, title, and director, respectively. (Remember that in Ruby, movie.title = "Doctor Strangelove" is actually a disguised call to the method title=—the same as movie.title=("Doctor Strangelove").)

Proud of himself, Bob shows the code to his older, more experienced colleague, Bill. Bill stares at the screen for a few seconds and proceeds to shatter Bob's pride into tiny little pieces. "There's a lot of duplication in this code," Bill says. "You have a movies table with a title column in the database, and you have a Movie class with an @title field in the code. You also have a title method, a title= method, and two "title" string constants. You can solve this problem with way less code if you sprinkle some metaprogramming over it."

Enter Metaprogramming

At the suggestion of his expert-coder friend, Bob looks for a metaprogramming-based solution. He finds that very thing in the Active Record library, a popular Ruby library that maps objects to database tables. After a short tutorial, Bob is able to write the Active Record version of the Movie class:

```
class Movie < ActiveRecord::Base
end
```

Yes, it's as simple as that. Bob just subclassed the ActiveRecord::Base class. He didn't have to specify a table to map Movies to. Even better, he didn't have to write boring, almost identical methods such as title and director. It all just works:

```
movie = Movie.create
movie.title = "Doctor Strangelove"
movie.title  # => "Doctor Strangelove"
```